





Nuværende Datafordeler

- ÷ Nuværende REST-tjenester med forskellig logik og struktur fra register til register
- ÷ Komplekst for registre at specificere REST-tjenester via DLS – og høj time-to-market for implementering heraf
- ÷ Meget manuelt arbejde ifm. implementering af nye og ændrede REST-tjenester
- ÷ Mange data, der ellers er på Datafordeleren, er ikke udstillet via REST-tjenester
- ÷ Udstillingens output skemaer følger ikke altid grunddatamodellen



Moderniserede Datafordeler

- + GraphQL-udstilling for alle entiteter, som anvendere har adgang til
- + Samme logik og struktur for opslag af data, uanset register
- + Ingen register-specifikation (DLS) af udstillings-skemaer, tjenesteparametre eller tjenestelogik
- + Autogenerering af implementering af nye og ændrede GraphQL-tjenester, med minimalt manuelt arbejde
- + Anvendere kan nøjes med at hente de data der er brug for
- ! Anvendere skal (som hidtil) forholde til sammenhæng mellem data ud fra grunddatamodellen



Finansministeriet har i forbindelse med Forslag til finanslov 2026 bevilget budget til paralleldrift på Datafordeleren frem til den 30. juni 2026. Det betyder, at de ikke-moderniserede tjenester skal være udfaset, og at paralleldrift vil ophøre 30. juni 2026.

<https://datafordeler.dk/vejledning/modernisering/>



- Kort repetition på GraphQL
 - Adgang til den fleksible opslagslogik
 - Det fleksible schema
 - Konstruktion af forespørgsler
 - Eksempler på anvendelse
 - Kendte begrænsninger
-



Jeg er **ANVENDER** på Datafordeleren - mine eksempler og erfaringer er skabt på grund af brug af services. Jeg kan svare på mange **HVORDAN** spørgsmål (men langt fra alle) - og bestemt ikke på **HVORFOR** spørgsmål.





Et **GraphQL-schema** definerer de typer og felter, du kan spørge om i API'et.

Schemaet fungerer som:

- En **kontrakt** mellem klient og server
- En **struktur** over alle data og deres relationer
- En **validering** af dine queries

Et schema består af:

- **type** – definerer objekttyper (f.eks. **Adresse**, **Vejstykke**)
 - **Query** – definerer tilgængelige (læse)operationer
-



- **Entitet:** Hver query skal specificere hvilken entitet den omhandler. Eksemplet kan være entiteten BBR_Bygning fra BBR og
 - **Endepunkt:** En query sendes til et GraphQL-endepunkt som et GET- eller POSTrequest.
 - **Standardfiltre:** En query kan indeholde standardfiltre der filtrerer data. Query'en
 - for eksempel et lighedsfilter (eq: "0550") på kommunekode-kolonnen
-



- **Geometriske filtre:** En query kan indeholde geometriske filtre, der filtrerer data på baggrund af geometri.

- **Bitemporale filtre:** En query skal også indeholde bitemporale filtre. For eksempel et bitemporalt point-in-time-filter (virkningstid: "2024-11-12T14:41:33Z") på virkningstidspunktet.
- **Paging :** En query kan også definere hvordan data pagineres. Returnerer de første 10 rækker (first: 10) givet filtreringen samt paginginformation for resultatsættet (angivet i "pageInfo").



```
query {  
  BBR_Bygning( # Entitet  
    where: { # Standardfiltre  
      kommunekode: { eq: "0550" }  
    }  
    virkningstid: "2024-11-12T14:41:33Z" # Bitemporalt filter  
    first: 12 # Paging  
  ) {  
    pageInfo { # Paginginformation  
      startCursor  
      endCursor  
      hasNextPage  
      hasPreviousPage  
    }  
    nodes {  
      id_lokalId  
      kommunekode  
      virkningsaktoer  
    }  
  }  
}
```




```
"data": {
  "BBR_Bygning": {
    "pageInfo": {
      "startCursor": "TURBd01ESTBOMk...d0t6QXdPakF3",
      "endCursor": "TURBd01ESTBOMkv0TkdkbF1...NHdNVF16TVRjd0t6QXdPakF3",
      "hasNextPage": true,
      "hasPreviousPage": false
    },
    "nodes": [
      {
        "id_lokalId": "0000247a-4beb-4e08-8217-d3246a66ffc3",
        "kommunekode": "0550",
        "virkningsaktoer": "Konvertering2017"
      },
      {
        "id_lokalId": "0000247a-4beb-4e08-8217-d3246a66ffc3",
        "kommunekode": "0550",
        "virkningsaktoer": "EksterntSystem"
      }
    ]
  }
}
```



- Som tidligere - det er ikke længere nok med en webbruger og en tjenestebruger.
- Hvis man skal have adgang til frie data, er en token nok, men den skal oprettes

Administration

Log på Datafordelerens Administration og opret de IT-systemer, der skal have adgang til at hente data via Fildownload (HTTPS) og GraphQL.

DATAFORDELER ADMINISTRATION

Bemærk at hvert testmiljø har egen Administration.

TEST 03

TEST 04

TEST 06

Bemærk: Du skal ansøge i Administration for at få adgang til beskyttede data. Under hvert IT-system kan du oprette API-keys eller OAuth Shared Secrets, som du skal indsætte i URL for at hente data via HTTPS. Du kan også oprette og administrere andre anvendere, der skal have adgang til IT-systemer eller Administration.



Velkommen til Datafordeler Administrationen

For at få adgang til data fra Datafordeleren skal du oprette dig som anvender med enten e-mail eller MitID. Derefter kan du logge ind i Datafordelerens Administration og oprette de IT-systemer, der skal have adgang til at hente data. Under hvert IT-system kan du oprette API-keys eller OAuth Shared Secrets, som du skal indsætte i URL for at hente data via HTTPS. Du kan også oprette og administrere andre anvendere, der skal have adgang til IT-systemer eller Administration. Du skal ansøge i Administration for at få adgang til beskyttede data. Bemærk at hvert testmiljø også har egen Administration.



IT-systemer

IT-systemer

Oversigt over oprettede IT-systemer med information om status og din rettighed til IT-systemet. Vælg et IT-system.

Navn ▾	Organisation ▾	Oprettet ▾	Rettighed ▾	Status ▾
<u>MortenTest</u>	Privat bruger	03-06-2025	Ejer	Aktiv

Opret



IT-systemer / IT-system

Opret API-Key

Navn*:

MortensTestKey

Opret

GEO
FORUM

IT-systemer / IT-system

Opret API-Key

Navn*:

[Opret](#)

API-Key er oprettet

API-Key:

[Kopier](#)

Kopier ovenstående API-Key. Den bliver kun vist en gang, og der kan gå op til 15 minutter før API-Key er aktiv.

[Tilbage til IT-system](#)

MortenTest

- [API Keys](#)
- [Shared Secrets](#)
- [Certifikater](#)
- [IP Adresser](#)
- [Deling af IT-System](#)

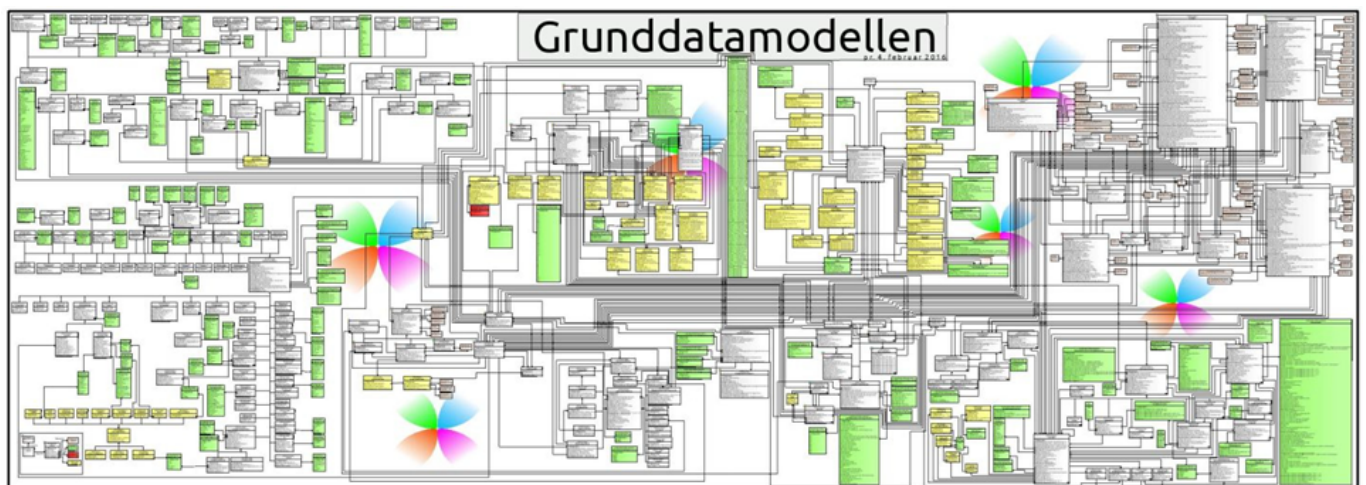
API-Keys

Opret API-Key for at give IT-systemet adgang til tjenester med frie data. Kopier og brug API-Key i URL til tjenesterne.

Navn ▾	Status ▾	Udløbsdato ▾
MortensTestKey	Aktiv	03-06-2027



- Realtions i grunddataprogrammet
- Endpoints
- Som text
- Som diagrammer



<https://www.version2.dk/artikel/fra-cpr-numre-til-vandloeb-dyk-ned-i-den-samlede-model-danske-grunddata>



[https://confluence.sdfi.dk/pages/viewpage.action?
pageId=187105434#GraphQL%C3%A5Datafordeleren-GraphQL-enderpunkter](https://confluence.sdfi.dk/pages/viewpage.action?pageId=187105434#GraphQL%C3%A5Datafordeleren-GraphQL-enderpunkter)

<https://confluence.sdfi.dk/display/DML/Fleksibel+opslagslogik>



GraphQL-tjenesterne udstilles gennem URL'er, der følger nedenstående form:

```
https://graphql.datafordeler.dk/<register>/<version>
```

Register	Forkortelse
Bygnings- og Boligregistret	BBR
Danmarks Adresseregister	DAR
Danmarks Fikspunktregister	FIKSPUNKT
DHM Højdekurver	DHMHøjdekurver
DHM Oprindelse	DHMOprindelse
Danske Stednavne	DS
Det Centrale Personregister	CPR
Det Centrale Virksomhedsregister	CVR
Ejendomsbeliggenhedsregistret	EBR
Ejendomsvurdering	VUR
Ejerfortegnelsen	EJF
GeoDanmark Vektor	GEODKV
Historiske kort og data	HISTKORT
Matriklen2	MAT
Skatteforvaltningens Virksomhedsregister	SVR

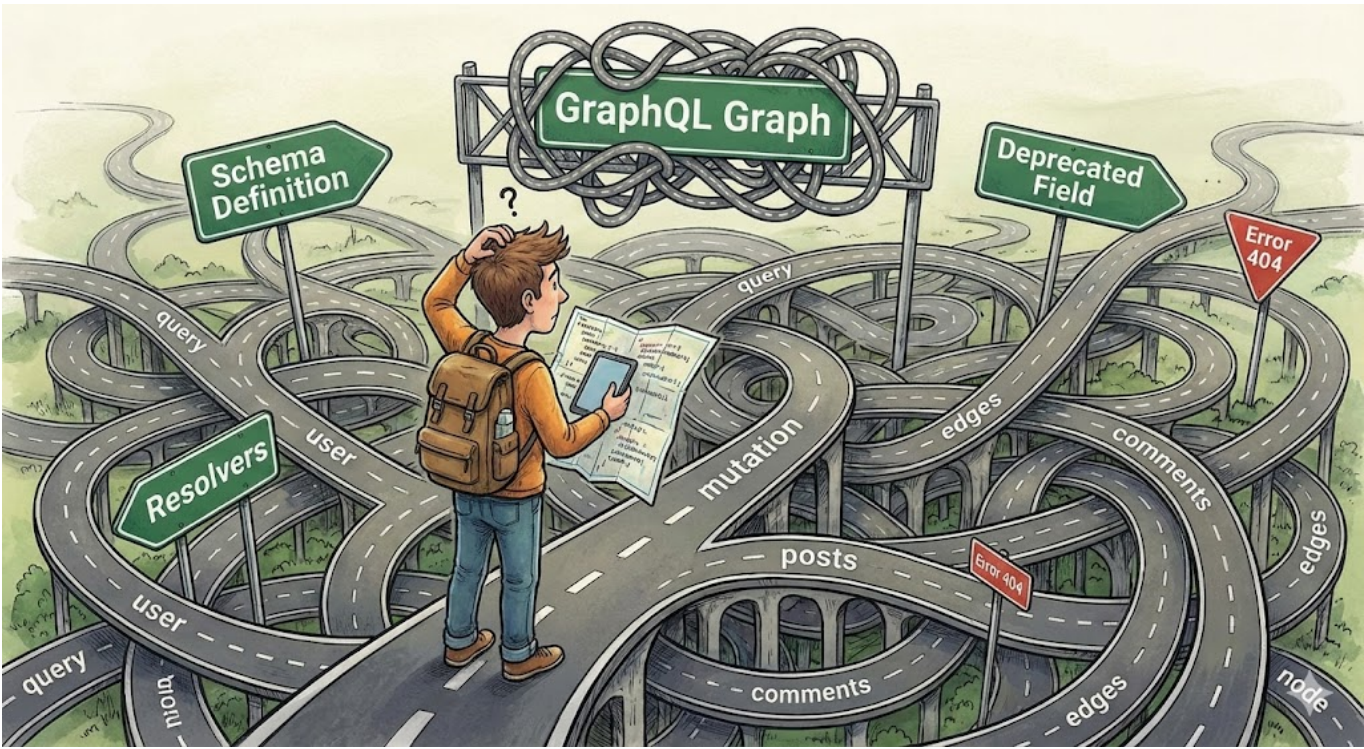
Udgører knap 74.000 linjer - indeholder representation af grunddata som edges, nodes and connections

```
72071 "The `Long` scalar type represents non-fractional signed whole 64-bit numeri
72072 scalar Long
72073
72074 "The `Short` scalar type represents non-fractional signed whole 16-bit numer
72075 scalar Short
72076
72077 "The `TimeSpan` scalar represents an ISO-8601 compliant duration type."
72078 scalar TimeSpan
72079
```



Query

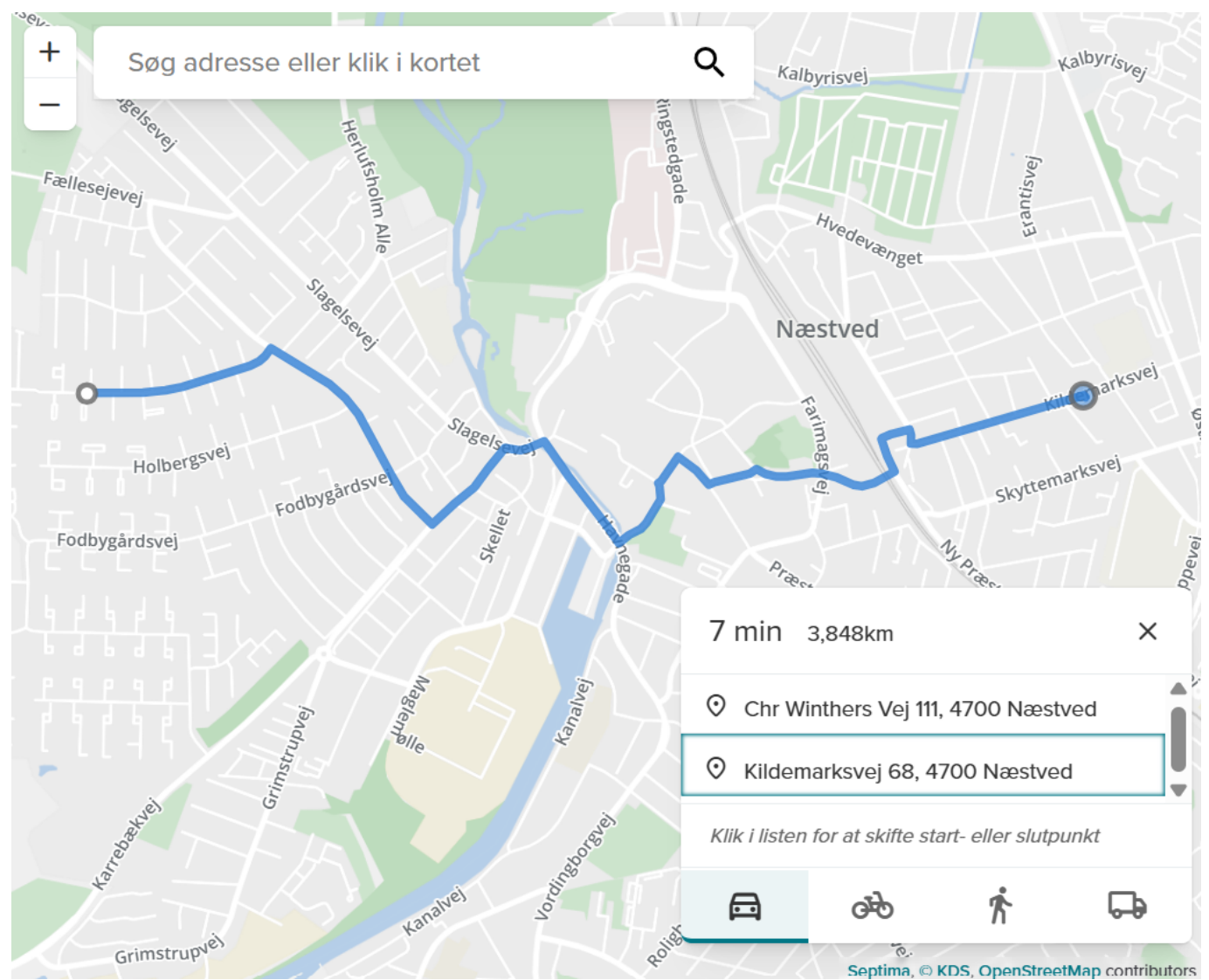
```
└─ DAR_Adresse(...) : DAR_AdresseConnection
  └─ nodes[] : DAR_Adresse
    └─ adresseHarHusnummer : DAR_HusnummerConnection
      └─ nodes[] : DAR_Husnummer
        └─ HusnummerHarAdgangspunkt : DAR_AdressepunktConnection
          └─ nodes[] : DAR_Adressepunkt
```





Som når vi navigerer, skal vi tænke vores GraphQL graph som et vejnet.

- Hvor starter du ?
- Har du stop på vejen ?
- Hvor skal du ende ?



Spørgsmålene kan relateres :

- Hvilke data er mit udgangspunkt ?
 - Har jeg en adresse, en matrikel eller en virksomhed jeg vil søge ud fra ?
 - Skal jeg bruge et udtræk af adresser med oplysninger, eller det virksomheder jeg vil søge ?
- Hvilke data skal jeg slutte med ?
 - BBR enheder for adresser ? Jordstykker for adresser ?
- Er der data 'på vejen' jeg skal bruge ?
 - BBR opgange på relationene mellem adresse og enhed ?



For at kunne skabe den forståelse, er en grafisk visning meget nemmere end teksten...

KDS har et webinar den 11/12 med dette specifikke formål - det vil jeg anbefale at man ser.

Modelsektariatet i KDS afholder et webinar, der giver Indblik i en metode til at identificere og forstå relationer i Grunddatamodellen. Tilmeld dig til webinarret via daf2@kds.dk.



- Viser i en browser
- Kan navigeres som et webkort
- Indeholder al information fra schemaet i et samlet overblik

Lad os se det engang...



GEO
FORUM

Jvf. det tidligere slide, så skal vi alligevel finde et 'startpunkt' - og det kan man med fordel så bruge de mindre diagrammer til.





- Når man har en ide om hvilket indhold man gerne vil have ud, og hvor man skal starte, så er opgaven at bygge forespørgslen op. Dette kan gøres på mange mere eller mindre smarte måder.



For mange vil første tanke nok være AI.

- ChatGPT kan læse schemaet (Det kræver dog en større plan)
 - Det kræver også noget tålmodighed, det går langsomt...



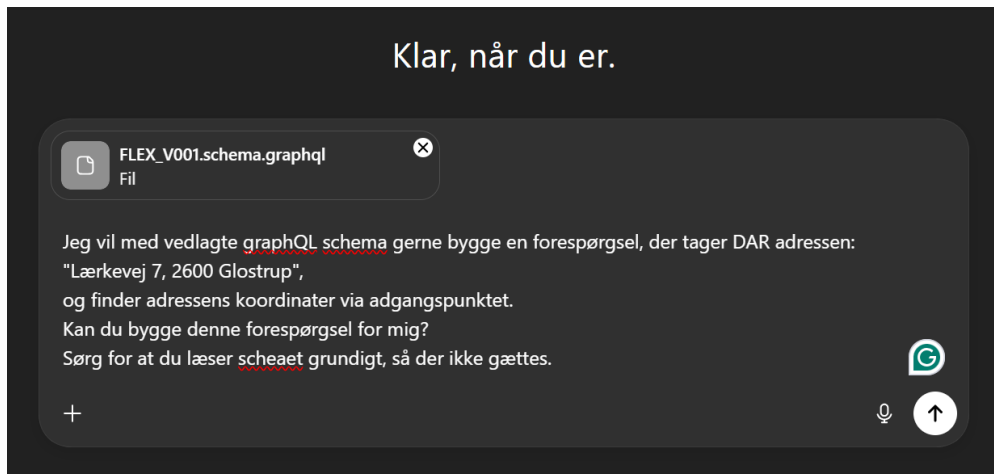
Jeg vil med vedlagte graphql schema gerne bygge en forespørgsel, der tager DAR adressen:

"Lærkevej 7, 2600 Glostrup",

og finder adressens koordinater via adgangspunktet.

Kan du bygge denne forespørgsel for mig?

Sørg for at du læser schemaet grundigt, så der ikke gættes.



Lad os se hvad der sker...



Den bedste mulighed jeg kender til, er i Postman, hvor man også kan give den schemaet...

- Lad os se hvordan det foregår
- Guiden 'Postman Guide til GraphQL' fra KDS ligger i mit Github Repo, den beskriver dette i detaljen



Lad os se nogle af 'mine' eksempler



- Max 20 joins pr. forespørgsel
- Ingen brug af alias'er
- Kun en root-node pr. forespørgsel



[Github.com/MFuglsang/graphql_datafordeleren](https://github.com/MFuglsang/graphql_datafordeleren)

Her er bla. en pdf med en lang række eksempler - langt flere end vi har vist her...



